Nombre		Matrícula
Ángel Uriel Lepe Rodriguez		2115842
Eder Abihú López Guzmán		2116464
Alexia Guadalupe Rodriguez Lara		2117441
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín	
Modulo Netacad: 3	**Actividad**: ADR VetCare_24	
Fecha: 16/10/2026		
Bibliografia (Formato APA)		

Objetivo:

El objetivo de los Architecture Decision Records es documentar de forma estructurada, transparente y justificada las decisiones arquitectónicas fundamentales tomadas para VetCare 24, registrando el contexto del negocio, las alternativas tecnológicas evaluadas y sus consecuencias, con el fin de preservar el histórico de diseño, facilitar la incorporación de nuevos desarrolladores al equipo y garantizar que cada patrón, límite de agregado o abstracción responda directamente a los requerimientos y principios de software de la materia.

ADR-001: Selección del Patrón de Lógica de Negocio (Domain Model)

Campo del ADR	Contenido que debe redactar el equipo
Título	**ADR-001: Selección del Patrón de Lógica de Negocio — Domain Model**
Estado	**Aceptada**
Contexto	**VetCare 24 administra operaciones clínicas críticas con complejas reglas cruzadas, incluyendo el bloqueo farmacológico por alergias (N-01), la interrupción/reordenamiento de la agenda por urgencias (N-03) y el seguimiento de constantes en hospitalización (N-04). Se requiere un patrón capaz de encapsular estados, invariantes y comportamientos evolutivos sin duplicar lógica.**
Alternativas	**• Transaction Script: Descartado porque organizaría la lógica en procedimientos lineales aislados. La validación de alergias y el recargo por urgencias se duplicarían en múltiples scripts, generando alto riesgo de inconsistencia al cambiar políticas clínicas.** **• Table Module: Descartado por organizarse alrededor de tablas de bases de datos. Una mascota o cita no son solo filas tabulares; poseen estados complejos y ciclos de vida que Table Module no puede gobernar adecuadamente.** **• Domain Model: Seleccionado porque modela el dominio mediante un grafo de objetos ricos (Mascota, AgendaCita, Hospitalización) que encapsulan sus propias reglas y protegen sus invariantes de manera nativa.**
Decisión	**Aceptar Domain Model como el patrón central de lógica de negocio para VetCare 24, combinándolo con la arquitectura táctica de Domain-Driven Design (DDD).**
Consecuencias	**• Beneficios: Encapsulamiento estricto de invariantes, alta mantenibilidad, trazabilidad con el negocio y bajo acoplamiento.** **• Costos/Riesgos: Mayor esfuerzo inicial de modelado y curva de aprendizaje para la construcción de agregados y repositorios.**
Trazabilidad	**N-01 a N-05; US-01 a US-07; CU-01 a CU-05**

ADR-002: Modelado de Límites de Agregados (Mascota, AgendaCita y Hospitalización)

Campo del ADR	Contenido que debe redactar el equipo
Título	ADR-002: Separación de Agregados Raíz y Referencias por Identificador
Estado	Aceptada
Contexto	Inicialmente se consideró agrupar la agenda de citas, el expediente de la mascota y los registros de hospitalización dentro de un solo gran objeto. Sin embargo, esto provocaría problemas graves de contención de datos y bloqueos transaccionales recurrentes durante la operación diaria.
Alternativas	• Un solo Agregado Monolítico (Mascota como raíz de todo): Descartado porque la actualización de la bitácora de hospitalización (cada 4-6h) o el reordenamiento de la agenda bloquearía todo el expediente clínico del animal, impidiendo consultas simultáneas o facturación. • Agregados Independientes referenciados por ID: Seleccionado. Se definen tres Aggregate Roots independientes: Mascota (id), AgendaCita (id) y Hospitalización (id), vinculados únicamente mediante sus identificadores (ej. mascotaId).
Decisión	Definir Mascota, AgendaCita y Hospitalización como Aggregate Roots independientes. Las relaciones entre ellos se gestionan exclusivamente por referencia de ID (mascotaId), acotando los límites de consistencia transaccional.
Consecuencias	• Beneficios: Elimina la contención de datos, permite transacciones cortas y eficientes, y facilita la escalabilidad y actualización independiente de cada área clínica. • Costos/Riesgos: La consistencia entre agregados debe coordinarse mediante eventos de dominio (Domain Events) de forma eventualmente consistente.
Trazabilidad	N-01, N-03, N-04; US-01, US-02, US-06; CU-01, CU-02, CU-04

ADR-003: Lógica Stateless Desacoplada mediante Domain Services

Campo del ADR	Contenido que debe redactar el equipo
Título	**ADR-003: Uso de Domain Services para Operaciones Multientidad y Sin Estado**
Estado	**Aceptada**
Contexto	**Operaciones como la evaluación de clientes duplicados en el registro o el cálculo de recargos por urgencia involucran múltiples entidades o requieren consultar colecciones globales antes de instanciar un objeto. Colocar esta lógica dentro de una entidad violaría el Principio de Responsabilidad Única (S de SOLID).**
Alternativas	**• Asignar la lógica a la Entidad Propietario o Mascota: Descartado porque obligaría a la entidad a conocer la base de datos completa de clientes o reglas externas de tarifas, sobrecargando su responsabilidad.** **• Implementar la lógica en la Capa de Aplicación: Descartado porque las reglas de duplicados y cálculo de recargos son reglas puras del negocio y quedarían desprotegidas en la capa de coordinación.** **• Domain Services Desacoplados (CalculadorTarifaEmergencia, EvaluadorDuplicadosPropietario): Seleccionado como componentes puros del dominio, sin estado interno (*stateless*).**
Decisión	**Crear Domain Services independientes dentro de la capa de Dominio para centralizar reglas de negocio que no pertenecen naturalmente a una sola entidad o aggregate.**
Consecuencias	**• Beneficios: Mantiene las entidades del dominio delgadas y enfocadas, protege el principio de Responsabilidad Única y facilita la reutilización y pruebas unitarias de las reglas.** **• Costos/Riesgos: Incrementa levemente el número de clases dentro de la capa de Dominio.**
Trazabilidad	**N-03, N-05; US-02, US-07; CU-02, CU-05**

ADR-004: Desacoplamiento de Persistencia mediante Repositorios e Inversión de Dependencias

Campo del ADR	Contenido que debe redactar el equipo
Título	ADR-004: Aislamiento del Dominio mediante Repositorios por Agregado y Unit of Work
Estado	Aceptada
Contexto	Para evitar que el modelo del dominio clínico quede acoplado a frameworks ORM o motores de base de datos SQL específicos, se requiere una estrategia de abstracción de almacenamiento.
Alternativas	• Acceso directo a base de datos / ORM desde el Dominio: Descartado porque contamina el núcleo del negocio con código técnico y dificulta las pruebas unitarias. • Patrón Repository por Aggregate Root + Unit of Work: Seleccionado. Se definen interfaces abstractas en la capa de Dominio (IMascotaRepository, IAgendaRepository) e implementaciones concretas en Infraestructura.
Decisión	Implementar interfaces de repositorio exclusivas para cada Aggregate Root y gestionar las transacciones mediante el patrón Unit of Work, aplicando Inversión de Dependencias (D de SOLID).
Consecuencias	• Beneficios: Dominio 100% puro y ejecutable sin base de datos; facilidad total para realizar pruebas unitarias automatizadas con Mocks. • Costos/Riesgos: Requiere escribir mapeadores entre las entidades del dominio y los modelos de la base de datos.
Trazabilidad	N-01 a N-05; CU-01 a CU-05; Principio D de SOLID